

レポート課題 5

05-191009 奥村真司

問 1

動作例

```
$ ocamllex lexer.mll
30 states, 1692 transitions, table size 6948 bytes
$ ocamlyacc -v parser.mly
$ ocamlc -c syntax.ml
$ ocamlc -c parser.mli
$ ocamlc -c lexer.ml
$ ocamlc -c parser.ml
$ ocamlc -c example.ml
$ ocamlc -o example syntax.cmo lexer.cmo parser.cmo example.cmo
```

考察

parser.output には LR 構文解析のオートマトンの各状態の情報などが書かれていた。arith_expr を expr に潰すと shift/reduce conflict が発生した。

問 2

動作例

```
# 5*3;;
- = 15
# 5*3+4;;
- = 19
# (5+3)*(2+7);;
- = 72
# (5+3)*(2+7)/(2*3);;
- = 12
# (4-2)*(5+3)*(2+7)/(2*3);;
```

考察

加減算と剰余算の優先順位を括弧なしで実現するために arith_expr と factor_expr にわけた。eval は EAdd とほぼ同様である。ただし EDiv で割る数が 0 のときは DivisionByZero 例外を吐くようにした。

問 3

動作例

```
# let a = 5;;
val a = 5
# let b = 100 in a+b;;
- = 105
# b;;
Fatal error: exception Eval.EvalErr
```

考察

ELet はもとの環境での式の評価結果を一時的定義の変数に束縛し、その変数で拡張した環境のなかで in 以降の式を評価してその評価結果と、拡張前の環境を返せば良い。トップレベルの変数定義では、もとの環境で式を評価して、その評価結果を束縛して環境を拡張し、それを返せばよい。

問 4

動作例

```
# (5+3=8)&&(2+9=10);;
- = false
# (5+3=8)&&(2+9=11);;
- = true
# (5+3=8)^(2+9=11);;
- = false
# (5+3=8)^(2+9=10);;
- = true
# (5+3=9)||(2+9=10);;
- = false
# false||true;;
- = true
# 5+3=8 && true;;
Fatal error: exception Parsing.Parse_error
-----
# (5+3=8) && true;;
- = true
# ((5+3=8)&&(2*3=6))&&(2+4=3*2);;
- = true
# 5 && true;;
Fatal error: exception Eval.EvalErr
```

考察

C 言語のように優先度を $= < \&\& < ||$ にすることも考えたが、 $\text{and}(\&\&), \text{or}(||), \text{xor}(\wedge)$ は $=, <$ と同等のレベルの演算子とし、3 つ以上結合するには必ず括弧をつけなければならないような言語仕様にした。ブール演算に対応するにあたって整数値の比較のみだった $=$ をブール値の比較もできるように拡張した。

発展 1

動作例

```
# +++;;
Parsing.Parse_error
# ?
Failure Unknown Token: ?
# $
Failure Unknown Token: $
# 5/0;;
DivisionByZero
# yyy;;
Eval.Err
Unbound value yyy
```

考察

課題 4 から main.ml と eval.ml を改良した。EvalErr 例外に string をつけてより詳細なエラー情報をもたせた。そして main.ml の read_eval_print 内で try with 文で lexer, Parser, 評価器内の例外を補足して、例外を補足した場合は例外の情報を表示してから例外発生前の環境のままプロンプトを表示して次のコマンドを引き続き要求するようにした。

発展 2

動作例

```
# let x = 10
let y = x+1
let z = x*y;;
val x = 10
val y = 11
val z = 110
# let a = 5
let b = 3
let c = b+a;;
val a = 5
val b = 3
val c = 8
```

考察

let 宣言の列は let 宣言のリストとして CDecls of (name * expr) list とした。これに伴って parser.mly の let 式の文法を変更した。parser.mly の lets は 0 個以上の let 文を表している。コマンドの実行に関しては上の let 文から順に宣言して環境を拡張していくだけであるから、再帰で簡単にできる。次に、宣言後の表示について説明する。ここまでインタプリタの出力は main.ml 内で行われており、それ以外のところで表示するのはよろしくないと考えたので、main.ml 内から

のみ出力されるようにした。実際には main.ml に変更を加えておらず、Printf.printf に与える文字列と値 (id と v) を工夫した。上の動作例での x,y,z の let 式ならば,id を”val x = 10\nval y = 11\nval z”,v を 110 となるように eval_command_env を書いている。

発展 3

動作例

```
# let x = 10;;
val x = 10
# let x = 50
  and y = x*2;;
val x = 50
val y = 20
# let x = 10;;
val x = 10
# let x = 50
  and y = x*2
  in x+y;;
- = 70
# let a = 50
  and b = 20
  and c = a+b;;
Fatal error: exception Eval.EvalErr
-----
# let a = 50
  and b = 20
  and c = 40
  in a+b+c;;
- = 110
# a;;
Fatal error: exception Eval.EvalErr
```

考察

発展 2 を少し変更して実装した。発展 2 ではトップレベル定義 (CDecls) の評価の際に変数が定義されるごとに環境が拡張され、その環境で次の変数の式を評価していったが、今回は同時に定義されるのでその部分を全ての変数についてもとの環境で評価するように変更してやればよい。全ての変数をもとの環境で評価してから、環境を拡張して返す。

局所定義についても同様で、全ての変数をもとの環境で評価して、それらで拡張した環境で in 以降の式を評価してやればよい。定義された変数の情報の表示についても発展 2 と同様である。syntax.ml を見ればわかるが、単なる let 式は let...and... の and 以降が一つもないものとみなされている。